CSIDS: Detecting Mimicry Attacks using Network Context with Host Behavior in IoT Networks

Surja Sanyal, Manas Khatua
Department of Computer Science and Engineering
Indian Institute of Technology Guwahati
Guwahati, Assam, India
surja.sanyal@iitg.ac.in; manaskhatua@iitg.ac.in

Pratik Chattopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology (BHU) Varanasi
Varanasi, Uttar Pradesh, India
pratik.cse@iitbhu.ac.in

***Abstract*—Intrusion Detection Systems (IDSs) are often tasked with protecting Internet of Things (IoT) networks. Host-oriented mimicry attacks (HMA) mimic legitimate behavior to evade IDSs. While host-based IDS solutions miss the network context of the incoming packets, network-based IDS solutions do not capture the host behavioral impact of the packets, resulting in low accuracy in detecting HMA. Hybrid implementations succeed in reducing resource consumption through workload distribution, but struggle to maintain high detection accuracy as the network scales up. This work proposes a network Context Sensitive, hybrid IDS solution (`CSIDS`) that can detect HMAs with high accuracy while consuming fewer network and host resources. Extensive experimentation shows that `CSIDS` provides greater than 50% of computation savings in its network component and achieves an HMA detection rate of 97% while staying below the one-bit-per-packet state-of-the-art communication overhead.**

***Index Terms*—Internet of Things (IoT), IoT security, Intrusion Detection System (IDS), host-oriented mimicry attack (HMA), network context, computation reduction, resource consumption.**

I. Introduction

Over time, the Internet of Things (IoT) is becoming increasingly heterogeneous as more devices join the network. The augmentation of industrial control systems (ICS) with modern IoT technology has further increased the network's heterogeneity and greatly expanded the attack surface for attackers [1]. Advanced attacks, such as host-oriented mimicry attacks (HMA), mimic network and/or host activities to systematically evade detection by Intrusion Detection Systems (IDS) while carrying out malicious activities in the host devices. The combination of rule/signature-based and anomaly-based IDSs for detecting known and unknown attacks has been successful in IoT networks [2]. However, HMAs continuously evolve in their attack strategies and may go undetected by rule/signature-based IDSs [3], [4]. Decision-making and detection-strategy adaptation while the IDS is deployed can make a significant difference in how IDSs handle such continuously evolving threats [5]–[8]. Host-based IDSs (HIDS) [9] and network-based IDSs (NIDS) [10] focus on host behavior and network activity, respectively, for detecting attacks. However, NIDSs [10] are unable to detect HMAs if there are no network footprints for these. On the other hand, HIDSs [11] have low detection rates for HMAs. A majority of current hybrid IDS solutions use per-packet system call and program execution branch sequence tracking, with both packet and host analyzers placed on the host [12], [13]. They detect HMAs, but at a higher resource cost. Hybrid IDSs that split the per-packet network behavior tracking overhead to the router and the device behavior tracking to the host devices have succeeded in reducing the resource demand on the host devices [14], [15]. However, when the detection mechanism is pushed to the router for such hybrid IDSs, the data communication and computation overheads at the router become high [14]. Machine Learning (ML) based IDSs are promising solutions for anomaly-based attack detection [16]. The improved attack-detection capabilities and the advantages of an ensemble model-based approach are also studied [17]. Leveraging the temporal nature of attack data has given modern IDSs great capability to detect attacks that proceed gradually toward host exploitation to avoid detection [12], [18].

Such ML/DL-based detection strategies place the network under considerable computational and resource load. Multiple routes have been taken to reduce these loads on the network devices. Proper feature engineering and selection before detector training have a significant impact on reducing computational load on IDSs operating on resource-constrained networks, thereby avoiding the scalability issue of security solutions [19]. Sensor reading prediction using ML methods, enabling the sensor hardware to be turned off to save power, has been proposed [20]. The combination of supervised, semi-supervised, and unsupervised learning helps reduce the dimensionality of data generated by the network, thereby significantly reducing the IDS's computational overhead [21]. External-voltage-monitor-based IDSs use separate hardware for data collection and/or external processing boards for anomaly detection [9]. To the same end, the more intensive computations required by IDSs are pushed to edge/cloud devices or to resource-rich border routers to secure the network while preserving device resources [14], [22], [23]. In addition, smaller, quantized models that occupy less device memory have been developed to consume even less network resources while performing at par with their peers [12]. Architectural changes have also been proposed, ranging from model predictive control to estimate the future behavior of host devices [24], to ensemble detectors that reduce or replace resource-intensive DL model layers with less resource-intensive ones [17], [25]. Using statistical methods instead of ML/DL-based

detection can further reduce the device's computational load [9], [26].

Most of the aforementioned works focus on tracking per-packet system call sequences and program execution branch sequences to detect HMAs. This approach either increases the resource consumption rates of the host devices or those of the router. The former depletes the resource-constrained IoT network faster, while the latter increases the router's load, leading to scalability and network communication issues. It is worth noting that some of the aforementioned works focus on periodic network and host behavior analysis. However, they do not have high HMA detection rates. In brief, all such solutions have at least one issue, including detection accuracy, communication overhead, computation overhead, and resource consumption. Finally, the summary of the notable features of the existing works capable of detecting HMAs is presented in Table I.

This work proposes a network Context Sensitive hybrid IDS framework (CSIDS) to detect HMA with high accuracy while the computation and communication overheads remain low. CSIDS analyses host energy consumption periodically (i.e., window-based) rather than the per-packet system call and program branch sequence analysis performed by traditional IDSs. This results in fewer computation requirements over a time period as opposed to processing of hundreds of system calls invoked by each packet. CSIDS proposes an architectural design to further reduce the number of computations required to perform. In addition, CSIDS analyzes device behavior using the corresponding network behavior as context, thereby enabling detection of network footprints caused by HMAs. Lastly, CSIDS sends reduced context vectors to the host devices to reduce router-to-device communication such that it is in per with the traditional solutions. Simulation and analysis show that CSIDS achieves greater than 50% computation savings in its network component, with an HMA detection rate of 97% area under the curve (AUC), while keeping communication overhead below the one-bit-per-packet limit of traditional hybrid IDS solutions. The summary of the contributions of this article is as follows:

1) A periodic, window-based behavior monitoring scheme is proposed to lower the computation demands on the router and the host devices, thereby circumventing the per-packet analysis used by the state-of-the-art solution.
2) An architectural design (NETCON) is proposed to leverage network and device behavior in the anomaly detection process at host devices to increase HMA detection accuracy.
3) The proposed architectural design NETCON also reduces the router-level computations of the network component of the hybrid IDS during the detection phase.
4) A data postprocessing scheme (AMORT) is proposed to reduce communication overhead between the router and hosts, thereby maintaining amortized overhead below that of existing hybrid implementations.

The remainder of this article is organized as follows.

TABLE I
EXISTING WORKS IN HMA DETECTION IN IOT NETWORKS

Scheme	DR	RC	HBE	COR	CON	Type	DD
[9]	Low	Low	Periodic	NA	NA	HIDS	×
[10]	Low	Low	Packet	Medium	NA	NIDS	×
[12]	Medium	High	Packet	NA	NA	Hybrid	✓
[13]	Low	High	Packet	NA	NA	HIDS	×
[14]	Low	High	Packet	Low	High	Hybrid	×
Proposed	High	Low	Periodic	Low	Low	Hybrid	✓

NA = Not Available, DR = Detection Rate, RC = Resource Consumption, HBE = Host Behavior Extraction, COR = Computation Overhead on Router, CON = Communication Overhead on Network, DD = Distributed Detection.

Section II presents the proposed IDS framework. Section III presents the theoretical analysis and the evaluation results of the proposed solution. Section IV draws the conclusion.

II. PROPOSED SOLUTION

CSIDS reduces the load on host devices by performing periodic device energy consumption analysis rather than per-packet system call and program branch sequence analysis, as discussed in Section II-B2. To reduce the computational load on the router, CSIDS generates network context vectors using the proposed architectural design NETCON, rather than fully processing network packets to detect anomalies at the router itself, as discussed in Section II-B1. These network context vectors are to be communicated to the respective host devices for analysis with the corresponding device behavior, as discussed in Section II-B3. In addition, CSIDS uses the data postprocessing scheme AMORT to reduce the network-generated context vector traffic between routers and hosts (as discussed in Section II-B1) to align with existing hybrid IDS solutions. The attack model has been discussed in Section II-A.

A. Host-oriented Mimicry Attack (HMA) Model

HMAs are advanced attacks that mimic the network and/or host activities of benign devices to systematically evade detection by IDSs while carrying out malicious activities on host devices. HMAs interleave their malignant activities with legitimate behavior, making their detection challenging. In this work, the attacker is assumed to reside on the device and affect its performance, eventually affecting the network. For designing HMAs, the scheme of inserting attack code into randomly chosen positions within benign sequences of device behavior has been followed [27]. To reduce the chances of detection by anomaly detectors based on system calls and program branch sequences [12], [14], the attack code only modifies the values of local variables that alter the frequency of device communications. This increases the device's radio duty cycle, reducing network lifetime.

B. CSIDS Framework for HMA Detection

In the IDS framework CSIDS, the *Network Context Extractor* (NCE) uses incoming packets to extract network context. These context vectors are then transferred to the respective IoT devices, which are the destinations of these packets. In the meantime, the raw packets have reached their destination IoT devices. The behaviors of these devices in response to

these packets are the corresponding device behaviors for these incoming packets. The *Device Behavior Extractor* (DBE) captures these behaviors and periodically extracts the relevant feature vectors. These system behavior feature vectors, along with the relevant network context vectors, serve as inputs to the *Anomaly Detector* (AD), which consumes both and classifies the combined set of behaviors as benign or malicious. The NCE, DBE, and AD are discussed in detail below.

1) Network Context Extractor (NCE): The NCE is an ensemble of multilayer perceptron (MLP) [28] based autoencoders (AE) [29] with one input layer for the n network input features, one output layer of n features, and one bottleneck layer of $\lfloor\sqrt{n}\rfloor$ features. This architecture has been chosen for its fast, lightweight feature reconstruction capabilities [10]. Network activity has been analyzed using features similar to the ones selected in [12]. In this work, training has been performed in offline batches rather than in online mode. An aggregator MLP AE, namely, the *Context Generator* (CG), is created and trained using this combined array of latent vectors LV_N obtained from the ensemble. Using LV_N, the CG generates a context vector CV_N, which is one of the inputs of the AD on the host device. In the detection phase, this proposed architectural design, `NETCON`, bypasses the NCE decoders to generate the context vector using CG.

The `AMORT` scheme is applied to the contest vector to generate the reduced context vector RV_N. To do this, the latent vector from CG is divided into s batches of b batch size such that the total number of bits to be communicated, considering each latent value as a 4 byte float, does not exceed the total number of packets processed p. This ensures that the amortized communication remains at 1 bit per packet, as in the current hybrid solutions. For each batch, the mean of each feature is used to form a single latent vector. This reduced array of s latent vectors is then sent to the device having the process that is the destination for the original packet. In that device, the AD consumes this context vector for behavior classification.

2) Device Behavior Extractor (DBE): Traditionally, device system calls and program branch sequences for each network packet have been used to extract device behavior. This generates a large number of data points for the resource-constrained IoT devices to process. Therefore, the DBE of this work periodically implements the behavior extraction process based on the energy consumed by the device over a given time period for various activities, such as CPU computations and data transmissions. The DBE generates a concise device behavior vector and delivers the same to the AD for processing.

3) Anomaly Detector (AD): The AD is another ensemble of shallow AEs similar to that in the NCE. To capture the time-variant and sequential nature of the inputs, the MLP encoders in the ensemble AEs have been replaced with a layer of Long Short-Term Memory (LSTM) units, which have shown good performance for such data [30]. The latent space and the output layers of the ensemble AEs remain similar to those of the NCE. The features used for host behavior analysis are the energy consumed by the host during CPU computation and during packet transmission (Tx). The benign host activity X_H and the benign network context vector CV_N serve as inputs to the AD. These are combined using oversampling to form the combined benign behavior vector X_T. Using this benign data X_T, the features are clustered based on correlation, to form a featuremap F_H. Each feature cluster $f \in F_H$ is mapped to one AE in the ensemble. An aggregator MLP AE is created and trained using the reconstruction error vector EV_H from the ensemble. The maximum reconstruction error of the aggregator is selected as a threshold T_H and is one of the inputs to the AD's attack detection process. To reduce the computation demands of the IDS execution at host devices, this detection process is run periodically instead of the per-packet basis followed by existing hybrid solutions.

TABLE II
SIMULATION PARAMETERS.

Parameter Name	Value
Radio model	UDGM: distance loss
Topology	Random
Routing protocol	RPL lite
Transmission range	50 m
Router mote platform	Cooja mote
Client and server mote platform	Z1 mote

III. PERFORMANCE ANALYSIS

Simulations have been carried out using the Cooja simulator in Contiki-NG [31]. The total number of motes in the simulation was varied between 10 and 60, distributed randomly, with 10% of the motes as attack motes. The simulation was carried out for $9000-60000$ seconds with 20% attack duration. Other simulation parameters are mentioned in Table II. A predefined sample application in Contiki-NG, in which all nodes send predefined packets to the root node, was run in the Cooja simulator. These packets are sent in aperiodic intervals using the random multihop topology. Simulation was carried out to capture a total of 600000 network packets as a single packet capture (PCAP) file using the Wireshark packet capture tool [32], out of which 500000 were benign. Periodic host behavior was captured during simulation using the Energest module of the Cooja simulator and saved as text files. The number of data points collected for each host device varied between 4000 and 30000, depending on the number of motes in the simulation. The firmware for all the motes was modified to incorporate the programmatically initiated HMA attack.

The NCE and the AD modules of `CSIDS` were implemented using Python scripts, and the DBE was implemented using the Energest module in the Cooja simulator. The deep learning models of the NCE and AD modules were trained without GPU support on an AMD Ryzen 7 Pro 5250G CPU with 8 physical cores and 16 GB of RAM, clocked at 3200 MHz. All AEs were trained for 300 epochs with a batch size of 256 and a learning rate of 0.001. The LSTM AEs were trained using 32 timesteps. For comparison with a state-of-the-art IDS, the open-source Python implementation of Kitsune [10] was used to test its attack-detection prowess on the collected data. As per the Kitsune implementation, the supplied radio communication

data required at least 5000 records for the feature mapper and at least 50000 records to train the model. An additional minimum of 100000 records were required for the benign sample. The attack data must meet these collective minimum requirements for the Kitsune implementation to execute.

A. HMA Detection Results

Fig. 1, Table III, and Table IV present the simulation results for the implementations of three versions of the proposed hybrid IDS and the state-of-the-art NIDS Kitsune. Out of the three hybrid IDS implementations, the first one is the proposed framework CSIDS with both NETCON and AMORT implemented (CSIDS-FULL). The second one is the proposed framework CSIDS with only NETCON implemented (CSIDS-NET). This implementation demonstrates the effect of applying the communication-reduction scheme AMORT on detection accuracy. The third one, CSIDS-HOST, uses the LSTM-based host ensemble model of the CSIDS architecture in both its router and host device detectors, and detects attacks at both the router and the host device. NETCON and AMORT are not implemented in CSIDS-HOST. This implementation shows the effect of NETCON on the router's computation overhead and that of AMORT on the network's communication overhead. To make the comparison fair, all the hybrid IDS implementations use the same network and device behavior features for anomaly detection.

Fig. 1(a) presents the detection rates per attack attempt for the IDS implementations using Area Under Curve (AUC) and Equal Error Rate (EER) metrics. EER is defined as the point at which the false acceptance rate (FAR) and the false rejection rate (FRR) are equal. Fig. 1(a) shows that CSIDS-FULL has the highest detection AUC and the second highest EER values at 97% and 0.09, respectively. CSIDS-NET follows closely with marginal differences in the values of AUC and EER at 96% and 0.07, respectively. This proves that using the AMORT scheme does not have any significant negative impact on the detection capabilities of CSIDS-FULL. The CSIDS-HOST scheme has AUC and EER values of 79% and 0.24, respectively. Kitsune stays much closer to the random classification diagonal with AUC and EER values at 74% and 0.33, respectively. Thus, CSIDS-FULL shows a superior HMA detection rate compared to the other IDS implementations. Further, its HMA detection accuracy is unaffected by the communication reduction scheme AMORT.

Fig. 1(b) presents the communication overhead between the router and the host for the three hybrid IDS implementations. Since Kitsune is an NIDS, it does not incur such communication overheads. Fig. 1(b) shows that the amortized (per packet) communication overhead of the CSIDS-HOST implementation is constant at one bit per packet. The amortized communication overhead of the CSIDS-NET implementation is also constant at 32 bits per packet, due to the fixed context vector dimension. CSIDS-FULL shows the lowest average communication overhead of 0.973 bits per packet and maintains the same below one bit per packet throughout. Thus, CSIDS-FULL reduces network communication overhead to below that of traditional hybrid IDS solutions.

TABLE III
ROUTER COMPUTATION SAVINGS IN FLOPS

Router model Architecture	CSIDS-HOST	CSIDS-NET & CSIDS-FULL	Savings
MLP	59	28 (MLP only)	53%
LSTM	972		97%

TABLE IV
MOTE LEVEL ATTACK DETECTION

Mote level classification	CSIDS-HOST	CSIDS-NET & CSIDS-FULL	Improvement
Correctly classified	56%	78%	39%
Wrongly classified	44%	20%	55%
Attacked and detected	43%	85%	98%

Table III shows that CSIDS-FULL saves on average 53% of computations, in terms of floating point operations (FLOPs), done at the router for MLP ensemble AEs, and 97% of the same for ensemble AEs with LSTM encoders and MLP decoders. Therefore, CSIDS-FULL successfully reduces the total number of computations done at the router. Table IV presents the overall detection performance for the attacked motes instead of the detection of each attack attempt. It clearly shows that both CSIDS-FULL and CSIDS-NET have better detection rates than that of the CSIDS-HOST implementation, while Kitsune fails to detect the attacked motes. Since both CSIDS-FULL and CSIDS-NET use the NETCON design, they perform the same number of computations at the router (Table III) and have a marginal difference in detection accuracy (Fig. 1(a) and Table IV). However, they differ in their communication overheads as shown in Fig. 1(b). Thus, it is observed that Kitsune is unable to detect HMAs, whereas all three implementations of hybrid IDSs can. CSIDS-FULL has 98% higher motewise detection rate and 53% computation savings when compared to the CSIDS-HOST scheme, while maintaining router to host device communication overhead below one bit per packet as compared to CSIDS-NET (32 bits per packet) and CSIDS-HOST (one bit per packet). Note that, as Kitsune cannot detect HMA, the other results corresponding to Kitsune are not given in Table III and Table IV.

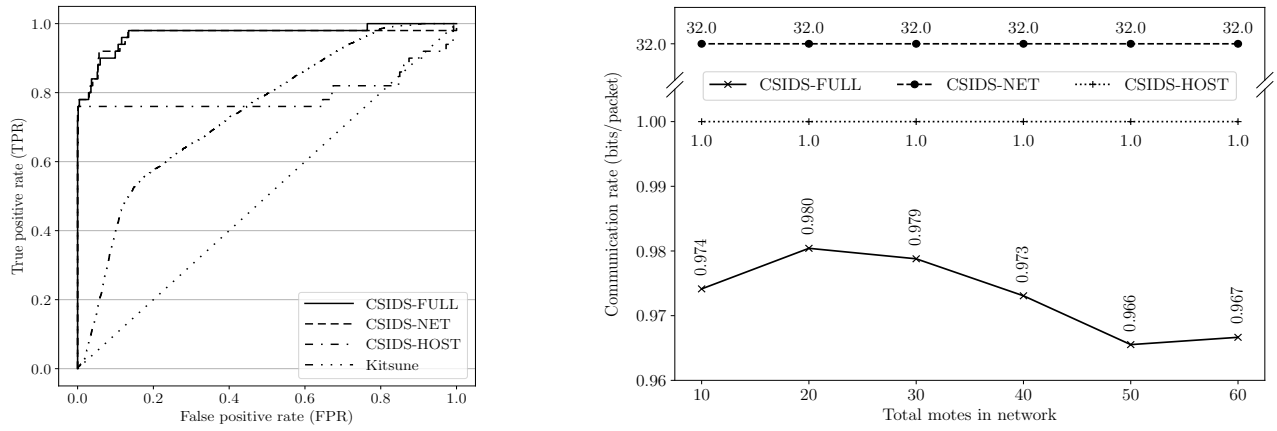
B. Theoretical Analysis

In this section, the theoretical analysis of the computation and communication overheads for CSIDS is presented and compared with those obtained from simulations.

1) Computation Overhead: The output value for a complete dense layer of n perceptrons with m inputs and activation function as $z(\cdot)$, can be represented by:

$$A = \sum_{j=1}^{n} z\left(y_j + \sum_{i=1}^{m} w_{ij} x_i\right). \tag{1}$$

where, x_i is the i^{th} input, y_j is the bias of the j^{th} perceptron of a layer, and w_{ij} is the weight for the i^{th} input to the j^{th} perceptron of the layer. Assuming that the function $z(\cdot)$ incurs



(a) (b)

Fig. 1. Simulation results for the execution of the NIDS Kitsune and three implementations of CSIDS. (a) AUC and EER scores. (b) Communication overhead.

a single floating point operation (FLOP), this gives us the total number of FLOPs as $N(A) = n \times [m + (m-1) + 1 + 1] = n(2m+1)$. To obtain the number of operations in the encoder and decoder parts of an AE, we only need to swap the values of m and n. Thus, we have $N(E) = m(2n+1)$ and $N(D) = n(2m+1)$, where we have assumed m $(1 \leq m \leq n)$ is the number of perceptrons in the bottleneck layer of the AE. Since, in the proposed architecture, the dimension of the latent space is the square root of that of the input space, we have $m = \lfloor \sqrt{n} \rfloor$. Using this, we compute the savings in computation operations that omitting the decoder provides.

$$N_s = \frac{N(D)}{N(E)+N(D)} = \frac{n(2m+1)}{m(2n+1)+n(2m+1)} = \frac{2mn+n}{4mn+m+n} \geq 50\%. \tag{2}$$

Thus, we have at least 50% savings in router computations when using CSIDS-FULL. The comparison of the simulation results with the theoretical estimate is presented in Table V, where we see that omitting the decoders from the network ensemble MLP AEs yields an average of 53% in computation savings, which agrees with the estimated $\geq 50\%$.

2) Communication Overhead: In contrast to the single-bit detection results generated by CSIDS-HOST, CSIDS-FULL generates latent context vectors at the router to be communicated to host devices. This incurs communication overheads for the proposed scheme. The amount of data to be communicated has been reduced to mitigate this overhead in resource-constrained networks. If the latent context vector (CV_N) for p packets has a features each, then the total number of bits for this vector, assuming q bits per float, is given by $|CV_N| = p \times a \times q$. However, for p network packets, the communication overhead during attack detection for CSIDS-HOST is only $p \times 1 = p$ bits. To achieve this as the upper limit on the communication overhead of CSIDS-FULL, the p packets are divided into s batches of size b. To represent each batch, the mean value of the b vectors for each feature is chosen for communication. Thus, only $s \times a \times q$ bits are communicated. The values of s and b are obtained as:

$$s = \left\lfloor \frac{p}{a \times q} \right\rfloor, \text{ and } b = \left\lceil \frac{p}{s} \right\rceil, \text{ and } s = \left\lceil \frac{p}{b} \right\rceil. \tag{3}$$

Here, the number of batches s is updated after calculating the ceiling of b to ensure that the total number of bits communicated by the router to the host device for p packets remains as close as possible to p, but capped at p. Therefore:

$$|RV_N| = q \times a \times s \leq \frac{q \times a \times p}{q \times a} = p. \tag{4}$$

Therefore, CSIDS-FULL incurs an amortized (per packet) communication cost of $|RV_N|/p \leq 1$ bit, at par with that of CSIDS-HOST. The comparison of the simulation results with the theoretical estimation is presented in Table V. It shows that the mean communication overhead is 0.973 bits per packet, agreeing with the estimated value of ≤ 1.

TABLE V
COMPARISON OF THEORETICAL AND SIMULATION RESULTS

Metric	CSIDS implementation			Theoretical Estimation	Results in Simulation
	HOST	NET	FULL		
Computation (FLOPs)	59	28	28	$\geq 50\%$ reduction	53% reduction
Communication (bits per packet)	1	32.0	0.973	≤ 1	< 1

IV. CONCLUSION

It is observed that existing IDS solutions that detect HMAs either have low attack detection rates or high resource consumption. These issues arise from the resource-constrained nature of IoT networks. To improve the IDS detection rate, greater computational loads are often placed on the router and host devices. This increased load depletes the network's resources. To handle such issues, a distributed, network Context Sensitive, hybrid IDS solution (CSIDS) is proposed. CSIDS uses NETCON, the proposed architectural design, which relies solely on network context extracted from network packets

for anomaly detection at host devices. NETCON reduces the computations performed by its network component by more than 50%. In addition, CSIDS bypasses per-packet device behavior tracking in favor of periodic tracking to reduce the number of data points processed by the anomaly detector without compromising detection accuracy.

Furthermore, to keep a check on the traffic load on the network, CSIDS uses AMORT, the proposed network context post-processing scheme, to reduce its communication overhead and match that of traditional hybrid IDS solutions. Extensive simulations and theoretical analyses confirm that CSIDS achieves an HMA detection rate of 97% AUC and reduces its computations performed at the router by 53% as against those of the traditional hybrid solutions, while having less than one bit per packet average communication overhead, which is in line with the conventional solutions.

This work assumes the attack is in the exploit phase, where the attacker affects device and network performance. In the future, work will be done assuming that the attack is in the preamble phase, the course of actions taken by the attacker on the way to take control of the device.

REFERENCES

[1] I. A. Khan, D. Pi, M. Z. Abbas, U. Zia, Y. Hussain, and H. Soliman, "Federated-SRUs: A federated simple recurrent units-based IDS for accurate detection of cyber attacks against IoT-augmented industrial control systems," *IEEE Internet of Things Journal*, 2022.

[2] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for internet of vehicles," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 616–632, 2021.

[3] G. Costa, A. Forestiero, and R. Ortale, "Rule-based Detection of Anomalous Patterns in Device Behavior for Explainable IoT Security," *IEEE Transactions on Services Computing*, 2023.

[4] H. El-Taj, F. Najjar, H. Alsenawi, and M. Najjar, "Intrusion detection and prevention response based on signature-based and anomaly-based: Investigation study," *International Journal of Computer Science and Information Security*, vol. 10, no. 6, pp. 50–56, 2012.

[5] Y. Yao, Y. Tian, X. Li, X. Yang, B. Zhao, and Y. Yang, "Q-Learning Based MEP Search Algorithm and Coverage Enhancement Strategy in IoT-Enabled Intrusion Detection," *IEEE Sensors Journal*, 2023.

[6] D. Jin, S. Chen, H. He, X. Jiang, S. Cheng, and J. Yang, "Federated incremental learning based evolvable intrusion detection system for zero-day attacks," *IEEE Network*, vol. 37, no. 1, pp. 125–132, 2023.

[7] R. R. dos Santos, E. K. Viegas, A. O. Santin, and V. V. Cogo, "Reinforcement learning for intrusion detection: More model longness and fewer updates," *IEEE Transactions on Network and Service Management*, 2022.

[8] J. Tao, T. Han, and R. Li, "Deep-reinforcement-learning-based intrusion detection in aerial computing networks," *IEEE Network*, vol. 35, no. 4, pp. 66–72, 2021.

[9] B. T. Beasley, G. D. O'Mahony, S. G. Quintana, A. Temko, and E. Popovici, "Lightweight Anomaly Detection Framework for IoT," in *Proc. of 31st Irish Signals and Systems Conference (ISSC)*, 2020, pp. 1–6.

[10] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: an ensemble of autoencoders for online network intrusion detection," *arXiv preprint arXiv:1802.09089*, 2018.

[11] D. Lightbody, D.-M. Ngo, A. Temko, C. Murphy, and E. Popovici, "Host-Based Intrusion Detection System for IoT using Convolutional Neural Networks," in *Proc. of 33rd Irish Signals and Systems Conference (ISSC)*, 2022, pp. 1–7.

[12] S. Ahn, H. Yi, Y. Lee, W. R. Ha, G. Kim, and Y. Paek, "Hawkware: network intrusion detection based on behavior analysis with ANNs on an IoT device," in *Proc. of 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.

[13] J. Seo, I. Bang, J. You, Y. Cho, and Y. Paek, "SBGen: a framework to efficiently supply runtime information for a learning-based HIDS for multiple virtual machines," *IEEE Access*, vol. 8, pp. 225 356–225 369, 2020.

[14] H. Kim, S. Ahn, W. R. Ha, H. Kang, D. S. Kim, H. K. Kim, and Y. Paek, "Panop: Mimicry-Resistant ANN-Based Distributed NIDS for IoT Networks," *IEEE Access*, vol. 9, pp. 111 853–111 864, 2021.

[15] P. Bhale, D. R. Chowdhury, S. Biswas, and S. Nandi, "OPTIMIST: lightweight and transparent IDS with optimum placement strategy to mitigate mixed-rate DDoS attacks in IoT networks," *IEEE Internet of Things Journal*, 2023.

[16] U. Sabeel, S. S. Heydari, K. El-Khatib, and K. Elgazzar, "Unknown, Atypical and Polymorphic Network Intrusion Detection: A Systematic Survey," *IEEE Transactions on Network and Service Management*, 2023.

[17] S. D. Roy, S. Debbarma, and A. Iqbal, "A decentralized intrusion detection system for security of generation control," *IEEE Internet of Things Journal*, vol. 9, no. 19, pp. 18 924–18 933, 2022.

[18] C. Yin, S. Zhang, J. Wang, and N. N. Xiong, "Anomaly detection based on convolutional recurrent autoencoder for IoT time series," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 52, no. 1, pp. 112–122, 2020.

[19] L. Zhiqiang, Z. Jiangbin, W. Sifei, L. Zhijun, M. Asim, Y. Zhong, Y. Chen *et al.*, "Intrusion detection using hybrid enhanced CSA-PSO and multivariate WLS random-forest technique," *IEEE Transactions on Network and Service Management*, 2023.

[20] R. Laidi, D. Djenouri, and I. Balasingham, "On predicting sensor readings with sequence modeling and reinforcement learning for energy-efficient IoT applications," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 52, no. 8, pp. 5140–5151, 2021.

[21] M. Abdel-Basset, H. Hawash, R. K. Chakrabortty, and M. J. Ryan, "Semi-supervised spatiotemporal deep learning for intrusions detection in IoT networks," *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 12 251–12 265, 2021.

[22] I. Hafeez, M. Antikainen, A. Y. Ding, and S. Tarkoma, "IoT-KEEPER: Detecting malicious IoT network activity using online traffic analysis at the edge," *IEEE Transactions on Network and Service Management*, vol. 17, no. 1, pp. 45–59, 2020.

[23] Y.-C. Lai, D. Sudyana, Y.-D. Lin, M. Verkerken, L. D'hooge, T. Wauters, B. Volckaert, and F. De Turck, "Task assignment and capacity allocation for ML-based intrusion detection as a service in a multi-tier architecture," *IEEE Transactions on Network and Service Management*, vol. 20, no. 1, pp. 672–683, 2022.

[24] S. Bergies, T. M. Aljohani, S.-F. Su, and M. Elsisi, "An IoT-based deep-learning architecture to secure automated electric vehicles against cyberattacks and data loss," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 9, pp. 5717–5732, 2024.

[25] E. Nowroozi, M. Mohammadi, E. Savaş, Y. Mekdad, and M. Conti, "Employing deep ensemble learning for improving the security of computer networks against adversarial attacks," *IEEE Transactions on Network and Service Management*, 2023.

[26] C. Yang, X. Cao, L. He, and H. Zhang, "Distributed multiple attacks detection via consensus AA-GMPHD filter," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 53, no. 12, pp. 7526–7536, 2023.

[27] J. E. Tapiador and J. A. Clark, "Masquerade mimicry attack detection: A randomised approach," *Computers & Security*, vol. 30, no. 5, pp. 297–310, 2011.

[28] M. Riedmiller and A. Lernen, "Multi layer perceptron," *Machine Learning Lab Special Lecture, University of Freiburg*, vol. 24, 2014.

[29] W. H. L. Pinaya, S. Vieira, R. Garcia-Dias, and A. Mechelli, "Autoencoders," in *Machine learning*. Elsevier, 2020, pp. 193–208.

[30] R. DiPietro and G. D. Hager, "Deep learning: RNNs and LSTM," in *Handbook of medical image computing and computer assisted intervention*. Elsevier, 2020, pp. 503–519.

[31] G. Oikonomou, S. Duquennoy, A. Elsts, J. Eriksson, Y. Tanaka, and N. Tsiftes, "The Contiki-NG open source operating system for next generation IoT devices," *SoftwareX*, vol. 18, p. 101089, 2022.

[32] C. Sanders, *Practical packet analysis: Using Wireshark to solve real-world network problems*. No Starch Press, 2017.